Kotlin Backend Developer.Professional

otus.ru

REC **Проверить, идет ли запись**

Меня хорошо видно && слышно?

Kotlin Multiplatform

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

План вебинара

- Знакомство
- Kotlin Multiplatform – общие сведения
- Kotlin/JVM
- Рефлексия

Цели вебинара

После занятия вы:

1. Создавать мультиплатформенные проекты на Kotlin
2. Создавать тесты в мультиплатформенных Kotlin-проектах
3. Использовать целевые платформы JVM и Native для Kotlin

Смысл

Зачем вам это уметь

1. Использовать Kotlin для различных сценариев кроссплатформенной разработки
2. Пользоваться возможностями KMP

Общие сведения

Информация о Kotlin Multiplatform

Kotlin/JS (IR-based)	Stable	1.8	
Kotlin/Native Runtime	Stable	1.9.20	
Kotlin/Native memory manager	Stable	1.9.20	
klib binaries	Stable	1.9.20	
Kotlin Multiplatform	Stable	1.9.20	
Kotlin/Native interop with C and Objective C	Beta	1.3	
CocoaPods integration	Stable	1.9.20	
Kotlin Multiplatform Mobile plugin for Android Studio	Beta	0.8.0	Versioned separately from the language

1. Основная задача технологии – упрощение и ускорение разработки кросс-платформенных проектов за счет уменьшения времени, потраченного на создание и поддержку кода для различных целевых платформ
2. На рынке мультиплатформенные библиотеки – новый стандарт, их немало, но пока очень много не мультиплатформенных либ

Сценарии использования Kotlin Multiplatform

- **Kotlin Multiplatform Mobile (Android и IOS приложения).** Повторное использование кода мобильными приложениями на разных платформах
- **Full-stack web-приложения**. Использование общей логики на сервере и в браузере
- **Нативные приложения:** serverless, интеграция с нативными библиотеками, экономия памяти и размера исполняемого файла
- **Мультиплатформенные библиотеки.** Общий код + платформенно-зависимый код => мультиплатформенная библиотека, можно использовать в кросс-платформенных проектах (ktor, koin (DI) etc)
- **Общий код для мобильных и веб-приложений**

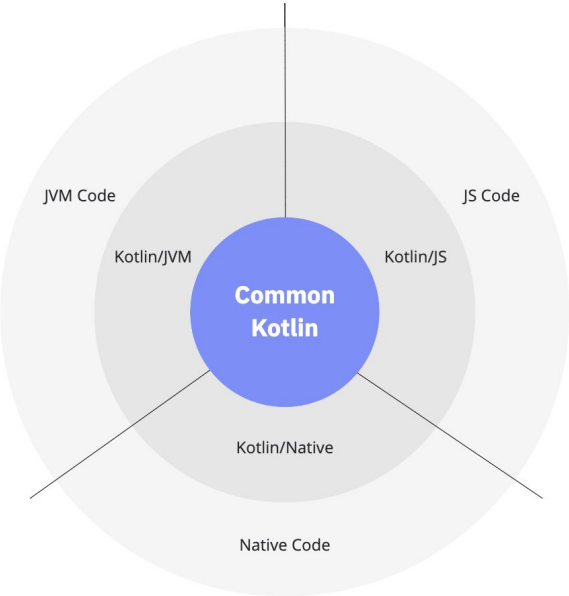
Принципы работы Kotlin Multiplatform

- **Common Kotlin**

Язык, основные библиотеки и инструменты. Работает на всех платформах. Может использовать multiplatform-библиотеки

- **Платформенно-зависимый код**

Специализированные версии Kotlin (Kotlin/JVM, Kotlin/JS, Kotlin/Native), обеспечивающие взаимодействие с кодом конкретной платформы



Некоторые целевые платформы, и их особенности и преимущества

Kotlin/JVM – в основном, сценарий серверных приложений. Выполняется в виртуальной машине Java, позволяет подключать любые библиотеки из богатой экосистемы JVM

Kotlin/JS – можно исподьзовать при создании клиентской части Web-приложений и интеграции с NodeJS. Есть DSL для HTML, CSS, React. Kotlin-код будет транспилирован в JS

Kotlin/Native – компиляция в двоичный код с интеграцией необходимого runtime, может взаимодействовать с нативным кодом на C / Objective C и другими нативными технологиями

Основные особенности multiplatform-проекта – использование multiplatform-плагина

- Позволяет задать и настроить целевые платформы
- Можно настраивать зависимости для каждой платформы отдельно

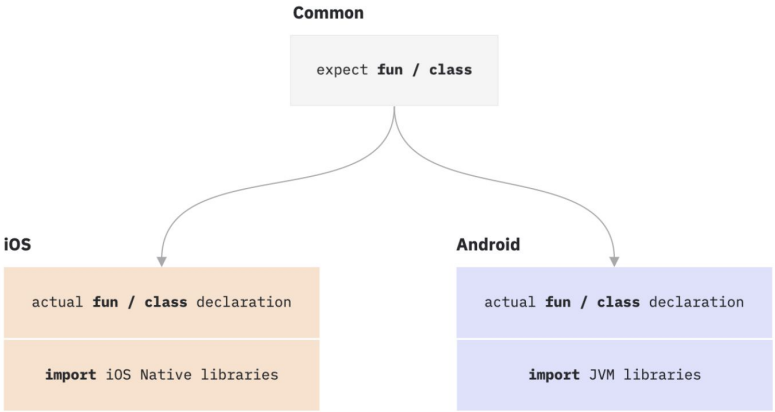
```
plugins {
    kotlin("multiplatform")
}
```

```
kotlin {
    jvm()
    js(IR)
    linuxX64("native")
}

sourceSets {
    val commonMain by getting
    val commonTest by getting
    val jvmMain by getting
    val jvmTest by getting
    val jsMain by getting
    val jsTest by getting
    nativeMain {}
    nativeTest {}
}
```

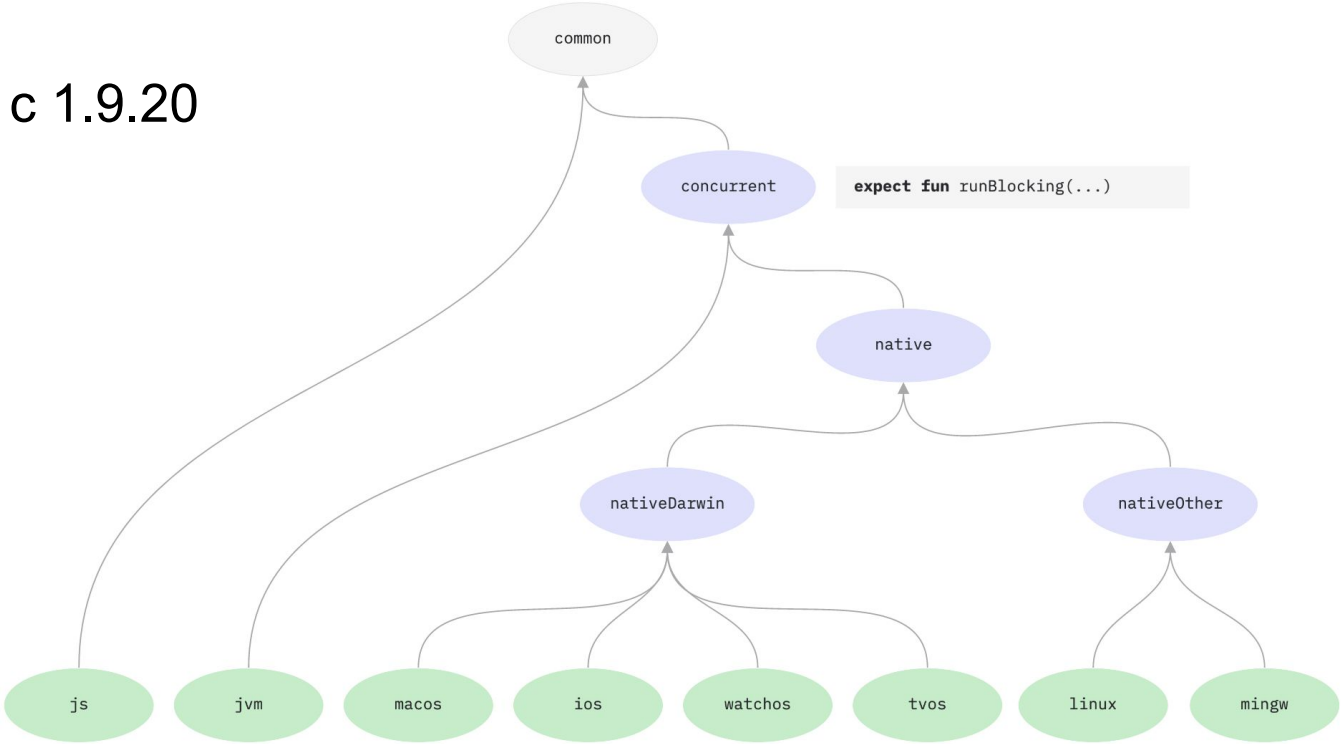
Основные особенности multiplatform-проекта – использование платформенных API

- При разработке может потребоваться задействовать платформенно-зависимый API
- Его реализация будет отличаться на разных платформа
- В Kotlin на этот случай существуют дополнительные ключевые слова
    - **expect** добавляется перед определяемой функцией
    - **actual** определяет в платформенном модуле
    - **external (JS, JNI)** говорит о том, что данная сущность реализована вне Kotlin



Иерархия платформ Kotlin

Начиная с 1.9.20



LIVE

Kotlin/JVM

Kotlin/JVM

- Основной сценарий использования, в рамках которого первоначально создавался Kotlin (Kotlin – лучшая Java, при этом доступно всё наследие экосистемы)
- Функциональная совместимость в обе стороны
  (Java -> Kotlin, Kotlin -> Java)
- Поддержка языка популярными фреймворками (Spring, Junit)
- Возможность использовать в одном проекте оба языка
- Плавная миграция с Java на Kotlin

Java из Kotlin: особенности работы

- Использование getters, setters как свойств
- Все объекты, которые «приходят» из Java – nullable, при этом поддерживаются nullability-аннотации
- Использование типов (например, примитивы Java (int) будут соответствовать kotlin.Int)
- Отсутствие checked-exceptions
- Отсутствие дефолтных значений
- И прочее – список объемный

Kotlin из Java: особенности работы

- Использование свойств как getters, setters
- Отдельно стоящие функции преобразуется в классы со static-методами
- Преобразование модификаторов доступа
- Возможность переопределять некоторые аспекты путём использования аннотаций в Kotlin-коде:
  - @JvmStatic – к примеру, возможность контроля генерации static-методов, определенных в объектах
  - @JvmName – даёт возможность управлять генерируемыми в Java именами
  - @JvmOverloads – возможность генерации перегруженных методов с различными вариантами значений по умолчанию
  - @Throws – указание исключений, которые может бросать метод
  - … https://kotlinlang.org/api/latest/jvm/stdlib/kotlin.jvm/

LIVE

Рефлексия и подведение итогов

Цели вебинара

Проверка достижения

1. Знаем как использовать целевые платформы для Kotlin (JVM, JS и Native)
2. Создавать многоплатформенные модули на Kotlin
3. Создавать и запускать тесты для JS / JVM / Native приложений

Тезисы

1.	Kotlin может использоваться как для серверной, так и для Full-Stack разработки, а также Multiplatform
2.	Есть возможность компиляции исходника на Kotlin во множество нативных платформ (Windows / Linux / MacOS / Android / iOS / WebAssembly). При этом для многих платформ реализуется функциональная совместимость (например, Kotlin -> Java, Java -> Kotlin)
3.	Модель expect – actual позволяет реализовать логику специальным платформенным кодом
4.	Использование external для доступа к платформенному коду (пример – DOM API, browser API)

Слайд с домашним заданием

1. создать три композитных проекта: (1) учебные модули; (2) плагин; (3) проектные модули
2. создать плагины Gradle для сборки подпроектов (1) на базе jvm; (2) на базе multiplatform
3. создать файл libs.versions.toml и зацепить за него версии зависимостей в плагинах

Сроки выполнения: **до начала Модуля 3**

Рефлексия

Остались ли у вас насущные вопросы по теме?

Какой аспект темы хочется показался самым сложным?

2 мин

Пишем в чат или говорим голосом

Список материалов для изучения

1. Документация по Kotlin Multiplatform: https://kotlinlang.org/docs/multiplatform.html
2. https://github.com/AAkira/Kotlin-Multiplatform-Libraries – список библиотек с поддержкой Kotlin Multiplatform
3. Calling Java from Kotlin
4. Calling Kotlin from Java
5. Use JavaScript code from Kotlin: https://kotlinlang.org/docs/js-interop.html
6. Use Kotlin code from JavaScript: https://kotlinlang.org/docs/js-to-kotlin-interop.html
7. Елена Лепилкина - Kotlin/Native: между LLVM и VM (доклад)
8. https://kotlinlang.org/docs/Build a full-stack web app with Kotlin Multiplatform .html

Следующий вебинар

Модуль 2: Расширенные возможности Kotlin

Интероперабельность с другими языками

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание! Приходите на следующие вебинары